

OOP in Java

BCA — Semester 3 — 2020 — Answer Sheet

Subject Code: CACS204

Group B

Attempt any SIX questions. [6 × 5 = 30]

2. Define OOP. Write the characteristics of OOP Language.

[5]

Answer: Object-Oriented Programming (OOP):

OOP is a programming paradigm that organizes software design around data (objects) rather than functions and logic. An object is an instance of a class that contains data (attributes) and methods (behaviors).

Characteristics of OOP Language:

- 1. Class:** A blueprint or template that defines the properties and behaviors of objects.
- 2. Object:** An instance of a class that encapsulates data and methods.
- 3. Encapsulation:** Wrapping data and methods together into a single unit (class) and hiding internal details from the outside world.
- 4. Inheritance:** The mechanism by which one class acquires the properties and methods of another class, promoting code reusability.
- 5. Polymorphism:** The ability to take multiple forms. It allows methods to perform different operations based on the object that invokes them (method overriding and overloading).
- 6. Abstraction:** Hiding complex implementation details and showing only essential features to the user.

3. Explain the operators available in Java programming.

[5]

Answer: Operators in Java:

1. **Arithmetic Operators:** + (addition), - (subtraction), * (multiplication), / (division), % (modulus)
2. **Relational Operators:** == (equal to), != (not equal to), > (greater than), < (less than), >= (greater than or equal to), <= (less than or equal to)
3. **Logical Operators:** && (logical AND), || (logical OR), ! (logical NOT)
4. **Assignment Operators:** = (simple assignment), +=, -=, *=, /=, %= (compound assignment)
5. **Increment/Decrement Operators:** ++ (increment by 1), -- (decrement by 1)
6. **Bitwise Operators:** & (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), >> (right shift), >>> (unsigned right shift)
7. **Conditional (Ternary) Operator:** condition ? expression1 : expression2
8. **InstanceOf Operator:** Checks whether an object is an instance of a particular class.

4. Define loop. Write a java program to print first n prime numbers.

[5]

Answer: Loop:

A loop is a control structure that allows a block of code to be executed repeatedly based on a condition. Java provides three types of loops: for, while, and do-while.

Program to print first n prime numbers:

```
import java.util.Scanner; public class PrimeNumbers { public static void main(String[] args) { Scanner sc = new Scanner(System.in); System.out.print("Enter n: "); int n = sc.nextInt(); int count = 0; int num = 2; while (count < n) { boolean isPrime = true; for (int i = 2; i <= num / 2; i++) { if (num % i == 0) { isPrime = false; break; } } if (isPrime) { System.out.print(num + " "); count++; } num++; } }
```

5. Differentiate between abstract class and interface with suitable example.

[5]

Answer: Abstract Class vs Interface:

Abstract Class	Interface
Declared with abstract keyword	Declared with interface keyword
Can have both abstract and concrete methods	Can only have abstract methods (before Java 8); default/static methods allowed since Java 8
Can have instance variables	Variables are public, static, and final by default
Supports constructors	No constructors
A class can extend only one abstract class	A class can implement multiple interfaces
Used when classes share common code	Used to define a contract for classes

Example:

```
abstract class Animal { abstract void sound(); void sleep() { System.out.println("Sleeping"); } } interface Drawable { void draw(); }
```

6. Define access modifier. Explain access modifiers in java with example.

[5]

Answer: Access Modifier:

Access modifiers in Java define the visibility or accessibility of classes, methods, and variables. They control which parts of the program can access a particular member.

Types of Access Modifiers in Java:

1. **private:** Accessible only within the same class. Most restrictive.
2. **default (no modifier):** Accessible within the same package.
3. **protected:** Accessible within the same package and by subclasses in other packages.
4. **public:** Accessible from anywhere. Least restrictive.

Example:

```
public class Demo { public int a = 10; // accessible everywhere private int b = 20; // accessible only in Demo
protected int c = 30; // accessible in package + subclasses int d = 40; // default - accessible in package only }
```

7. Define exception. Explain exception handling mechanism in java with example.

[6]

Answer: Exception:

An exception is an event that disrupts the normal flow of program execution. It is an object that is thrown at runtime when an error occurs.

Exception Handling Mechanism in Java:

Java uses the following keywords for exception handling:

1. **try:** Block of code that might throw an exception.
2. **catch:** Block that handles the exception thrown in the try block.
3. **finally:** Block that always executes, regardless of whether an exception occurred.
4. **throw:** Used to explicitly throw an exception.
5. **throws:** Declares exceptions that a method might throw.

Example:

```
public class ExceptionDemo { public static void main(String[] args) { try { int a = 10, b = 0; int result = a / b; }
catch (ArithmeticException e) { System.out.println("Cannot divide by zero: " + e.getMessage()); } finally {
System.out.println("Finally block executed"); } } }
```

8. Write short note on (Any Two):

- a) final keyword
- b) Collection class
- c) JDBC

[5]

Answer: a) final keyword:

The final keyword in Java is used to restrict modification:

- **final variable:** Cannot be changed once initialized (constant).
- **final method:** Cannot be overridden by subclasses.
- **final class:** Cannot be extended (inherited).

b) Collection class:

The Collection framework in Java provides a set of interfaces and classes to store and manipulate groups of objects:

- **List:** Ordered, allows duplicates (ArrayList, LinkedList)
- **Set:** Unordered, no duplicates (HashSet, TreeSet)
- **Map:** Key-value pairs (HashMap, TreeMap)
- **Queue:** FIFO structure (PriorityQueue, LinkedList)

c) JDBC (Java Database Connectivity):

JDBC is a Java API that enables Java programs to interact with databases:

- **DriverManager:** Manages database drivers
- **Connection:** Establishes connection to database
- **Statement:** Executes SQL queries
- **ResultSet:** Stores query results

Steps: Load driver → Create connection → Create statement → Execute query → Process results → Close connection

Group C

Attempt any TWO questions. [2 × 10 = 20]

9. a) Write a program to create and use java package.
b) Define thread. Explain the life cycle of thread.

[10]

Answer: a) Program to create and use Java package:

```
// File: MyPackage.java package mypack; public class MyPackage { public void display() {  
System.out.println("Hello from mypack!"); } } // File: TestPackage.java import mypack.MyPackage; public class  
TestPackage { public static void main(String[] args) { MyPackage obj = new MyPackage(); obj.display(); } }
```

b) Thread:

A thread is the smallest unit of execution within a process. Java supports multithreading, allowing multiple threads to run concurrently.

Life Cycle of a Thread:

1. **New:** Thread is created but not started.
2. **Runnable:** Thread is ready to run, waiting for CPU time.
3. **Running:** Thread is currently executing.
4. **Blocked/Waiting:** Thread is temporarily inactive (waiting for a resource or another thread).
5. **Terminated:** Thread has completed execution or been stopped.

10. a) Write a program to sort name of any five cities in ascending order.
b) Define polymorphism. How do we achieve polymorphism in java explain with example?

[10]

Answer: a) Program to sort names of five cities:

```
import java.util.Arrays; import java.util.Scanner; public class SortCities { public static void main(String[] args) {  
Scanner sc = new Scanner(System.in); String[] cities = new String[5]; System.out.println("Enter 5 city  
names:"); for (int i = 0; i < 5; i++) { cities[i] = sc.nextLine(); } Arrays.sort(cities); System.out.println("Cities in  
ascending order:"); for (String city : cities) { System.out.println(city); } } }
```

b) Polymorphism:

Polymorphism means "many forms." It allows objects of different classes to be treated as objects of a common superclass, and a single method can behave differently based on the object that calls it.

Types of Polymorphism in Java:

1. **Compile-time (Method Overloading):** Same method name with different parameters.
class MathOp { int add(int a, int b) { return a + b; } double add(double a, double b) { return a + b; } }
2. **Runtime (Method Overriding):** Subclass provides a specific implementation of a method declared in the parent class.
class Animal { void sound() { System.out.println("Animal sound"); } } class Dog extends Animal { void sound() {
System.out.println("Bark"); } }

11. a) Differentiate between java AWT and java Swing. Explain the different types layout managers in java GUI programming.

b) Write a java GUI program to calculate square of entered number.

[10]

Answer: a) AWT vs Swing:

AWT	Platform-dependent (heavyweight)	Platform-independent (lightweight)	Uses native OS components
Swing	Uses pure Java components	Package: javax.swing	Less components and features
	More components and richer features	Not customizable	Highly customizable (look and feel)

Layout Managers in Java GUI:

1. **FlowLayout:** Components arranged left-to-right, top-to-bottom.
2. **BorderLayout:** Components placed in NORTH, SOUTH, EAST, WEST, CENTER.
3. **GridLayout:** Components arranged in a grid of rows and columns.
4. **GridBagLayout:** Flexible grid layout with varying component sizes.
5. **CardLayout:** Multiple panels stacked like a deck of cards.
6. **BoxLayout:** Components arranged vertically or horizontally in a box.

b) Java GUI program to calculate square:

```
import javax.swing.*; import java.awt.*; import java.awt.event.*; public class SquareCalculator extends JFrame
{ JTextField numField, resultField; JButton calcBtn; public SquareCalculator() { setTitle("Square Calculator");
setSize(300, 150); setLayout(new GridLayout(3, 2)); add(new JLabel("Enter any number:")); numField = new
JTextField(); add(numField); calcBtn = new JButton("Calculate"); add(calcBtn); add(new JLabel("Square of
entered number:")); resultField = new JTextField(); resultField.setEditable(false); add(resultField);
calcBtn.addActionListener(new ActionListener() { public void actionPerformed(ActionEvent e) { double num =
Double.parseDouble(numField.getText()); resultField.setText(String.valueOf(num * num)); } });
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); setVisible(true); } public static void main(String[] args)
{ new SquareCalculator(); }}
```

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.